Sentiment detection in Spanish twits

Miguel García Bohigues

April 2023

1 Introduction

This memory comprehends the word done under the task TASS2017: Sentiment analysis at Tweet level. Along its sections we will mention and explain the processes followed to generate the best classifier possible.

The structure of the memory will be as follows: First we will talk a bit about the data and the required pre-process in order to work with it. In the next section we will talk about linear classifiers and its performance with the data set. After that, we will describe the experiments done with deep neural networks, including different types of learning. Finally, we will conclude.

2 Data

As said before, the data set belongs to a past task (TASS2017). It is a collection of three "xml" files: one for training, one for validation and finally one for testing. The later is the only one which comes without the corresponding labels. Which, for this case are:

- NONE: means that the polarity of the twit couldn't be determined
- P: means that the twit has **positive** polarity
- N: means that the twit has **negative** polarity
- NEU: means that the twit shows a bit of positive and negative polarity

As said before, the data comes in XML format so we had to parse it to a tabular format easier to manipulate. Once done, we had something like what table 1 shows.

	id	label	text
0	768213876278165504	NONE	-Me caes muy bien -Tienes que ...
1	768213567418036224	N	@myendlesshazza a. que puto mal ...
2	768212591105703936	N	@estherct209 jajajaja la tuya y ...
3	768221670255493120	P	Quiero mogollón a @AlbaBenito99 ...
4	768221021300264964	N	Vale he visto la tia bebiendose ...

Table 1: Top 5 rows of training data

Having the tabular form, the following step was to tokenize the different twits. For that we have used a build-in tokenizer included in the NLTK package for python. As a result, this tokenizer provides us with the twits, but having split the special symbols from the words and with all the text in lower case.

In order to ease the classification we've developed a small vocabulary reducer. Its main goal is to substitute the mentions to other users, tags, webs, and so on; with a single word. This way, "@Manolito" will be represented in the text as "mención".

3 Linear Classifiers

As a first approach to solve the task we have decided to use linear models to classify the twits. Linear models are commonly used to split data in a provided vector space, but currently we only have sequences of words. In order to use this kind of models, we have applied a vectorization to out tokenized texts. From the different vectorizers that are available at the sklearn package, we have decided to use CountVectorizer. After applying this transformation, we now dispose of a data matrix with 43625 features for each text.

Additionally, we have decided to add two more columns to that matrix. These will correspond to the number of positive and negative words respectively. To achieve that we have used a polarity dictionary as a reference. To include word deviations we have extracted the root of the word by applying to each word a stemmer model.

With this input data, we have trained a simple linear support vector classification model with the regularization parameter $C = 0.1$ and we have obtained a macro-f1 score of 0.42.

	precision	recall	f1-score	support
N	0.61	0.77	0.68	219
NEU	0.22	0.06	0.09	69
NONE	0.38	0.21	0.27	62
P	0.61	0.68	0.64	156
accuracy			0.58	506
macro avg	0.45	0.43	0.42	506
weighted avg	0.53	0.58	0.54	506

Table 2: Demo SVC model

At this moment, we don't know if our classifier is performing well or not, so to check it we have developed a baseline classifier. This one is really simple, it labels a text as P if it contains more positive word count than negative. It labels the text N otherwise. In case that the positive and negative counts are the same, the text is labeled as NEU. Finally, if there's no count for neither polarity the label is NONE. With this dummy model we have obtained a macro-f1 score of 0.38

As we see, the result obtained with the first SVC model outperforms the basic classifier, so it gives us courage to test other models. To do so, we have developed an automation script to launch multiple experiments at the same time. Specifically, we have trained:

- Six SVC models with values of $C \in [1, 0.5, 0.1, 0.05, 0.01, 0.005]$.
- 24 SGD models with different loss functions, values of α ranging from 0.1 to 0.0001 and different penalties
- two decision tree models with different criteria and max mepth

- 12 logistic regression models with different solvers and values of C in the same range as in the SVC
- A simple perceptron
- Five ridge classifiers with $\alpha \in [1, 1/4, 1/8, 0.05, 0.01]$

Also, we have used different values of ngram prais: From considering only unigrams to consider all from unigrams to tetragrams (4-gram). The best result has been shown with 4-gram, so for all the experiments this is the combination we have selected.

From all the above listed 50 combinations, the best model has been the Perceptron. It has obtained a macro-f1 score of 0.44 as table 3 shows. We have obtained similar values (macro-f1=0.43) with a svc model with both $C = 1$ and a SGD model with loss perceptron and $\alpha = 0.05$, as table 4 shows.

	precision	recall	f1-score	support
N	0.61	0.72	0.66	219
NEU	0.35	0.19	0.25	69
NONE	0.29	0.19	0.23	62
P	0.61	0.65	0.63	156
accuracy			0.56	506
macro avg	0.46	0.44	**0.44**	506
weighted avg	0.53	0.56	0.54	506

Table 3: Best linear model: Perceptron

	SVC			SGD			
	precision	recall	f1-score	precision	recall	f1-score	support
N	0.60	0.76	0.67	0.59	0.77	0.67	219
NEU	0.32	0.09	0.14	0.32	0.16	0.21	69
NONE	0.36	0.21	0.27	0.25	0.18	0.21	62
P	0.59	0.67	0.63	0.63	0.59	0.61	156
accuracy			0.57			0.56	506
macro avg	0.47	0.43	0.43	0.45	0.42	0.43	506
weighted avg	0.53	0.57	0.54	0.53	0.56	0.53	506

Table 4: SVC vs SGD-Perceptron

As we can see in all three tables, the results are really similar although the Perceptron model has obtained a bit more f1-score. Thus, we hace decided to select this last one to generate our test labels. In order to obtain the best performance possible, we have re-trained the model with both the train and dev data.

4 Deep Learning Classifiers

For the deep learning classifiers we wanted to try different learning strategies, to compare their performance and also with the linear classifiers. This is specially interesting because of the lack of data. We only have available 1.5k samples to train which is very low for deep learning. So, knowing this, we have tried two approaches: An unsupervised version with zero-shot classification and a supervised version fine-tuning an existing model.

4.1 Zero-shot classification

The zero shot classification relies on generative language models such as BERT or GPT. In our case, we have decided to pick a cased BERT model for Spanish. Then we have passed to this model a simple template following the model guidelines: "Este twit tiene sentimiento label" and we have let the model to predict which word goes at "label" and label can be: 'positivo', 'negativo', 'neutro' or 'nada'.

With this trick, we have tried to guess our development set directly (as the training has lost its purpose since the model is already trained). The macro-f1 obtained as a result has been 0.39, slightly better than the dummy classifier mostly because of the lack of ability to predict the NONE tag, because the positive, negative and neutral f1-score is really close to the best linear model obtained.

	precision	recall	f1-score	support
N	0.69	0.62	0.66	219
NEU	0.20	0.42	0.27	69
NONE	0.00	0.00	0.00	62
P	0.63	0.63	0.63	156
accuracy			0.52	506
macro avg	0.38	0.42	0.39	506
weighted avg	0.52	0.52	0.51	506

Table 5: Zero-shot classification

To see if the macro-f1 improved if we use all de data available, we did another try with both train and dev sets concatenated, but the result was really similar. We also obtained a 0.39 value for the macro-f1 score and did not improved on the NONE tag.

Thinking about why failing only on the NONE tag, gave us the idea that maybe, the representation of 'nada' was too far from the meaning we were looking for. So we gave it another try with 'indefinido' instead of 'nada' and this really showed an improvement. As we can see at table 6, the model is capable of identifying some NONE labels and also as a result has improved its performance over the other labels.

	precision	recall	f1-score	support
N	0.71	0.62	0.66	219
NEU	0.18	0.37	0.24	69
NONE	0.26	0.07	0.12	62
P	0.64	0.64	0.64	156
accuracy			0.52	506
macro avg	0.45	0.43	0.41	506
weighted avg	0.55	0.52	0.53	506

Table 6: Zero-shot classification with 'indefinido' as label

4.2 Model fine-tuning

The idea that we are following in this approach is to take a model trained with a huge amount of data, attach a head to it and train the new model with the available data to classify the texts.

To do so we have selected BETO. A BERT model trained on a big Spanish corpus. To it we have attached a 4 neuron linear layer to obtain the logits and thus the prediction.

This model requires a special format on the data, so we have used its own tokenizer and data loader to satisfy the needs. In the process we had to translate the labels from text to number, so the model can perform the evaluation directly.

With the model and the data ready, we have trained the new model. We have done it with AdamW as the optimizer with a learning rate equals to 0.00005 and a linear scheduler.

With this system and playing a bit with the parameters, we have obtained the best macro-f1 ever since: 0.55, just as table 7 shows. There 0, 1, 2, 3 represent 'P', 'N', 'NEU', and 'NONE' respectively. Note that we have re-arranged the rows so it matches the order followed in the previous tables.

	precision	recall	f1-score	support
1.0	0.79	0.70	0.74	219
2.0	0.23	0.38	0.29	69
3.0	0.52	0.39	0.44	62
0.0	0.71	0.70	0.71	156
accuracy			0.62	506
macro avg	0.56	0.54	0.55	506
weighted avg	0.66	0.62	0.63	506

Table 7: BETO fine-tuned

As we can see, the biggest improvement has been achieved on the NONE label. There the model has obtained the highest value of all models with 0.26 points of macro-f1 more than the second best model. Furthermore, we see an improvement in all labels but one: NEU.

5 Conclusion

In this work we have tried different ways of building a classification system for sentiment analysis. We have seen from linear models to deep learning approaches. The best resulting model has been the one trained from an existing BERT and fine-tuning it to discriminate polarities. Other interesting conclusion is that the zero-shot classifier has been outperformed by the linear classifiers but has been really close to them.